Homework 5

Big Data Analytics (PETR)

Greyson Newton

In [9]:
```
import pandas as pd
df = pd.read_excel('hw5_150_samples.xlsx')
df
```

Out[9]:

	image_bank_number	Depth_max(nm)	Load(mN)	E(Gpa)	H(Gpa)
0	1	1760	99.913	62.7	2.07
1	2	1954	149.694	76.5	2.12
2	3	1968	149.817	78.9	2.81
3	4	2228	149.910	52.8	1.93
4	5	2035	149.953	71.1	2.40
...	...	...	...	...	...
145	146	3943	599.454	66.1	2.17
146	147	4473	598.946	62.3	2.34
147	148	4254	599.785	72.9	2.19
148	149	4048	599.453	72.4	2.19
149	150	4018	599.587	77.4	2.35

150 rows × 5 columns

In [10]:
```
import zipfile

def unzip(zip_file, destination):
    with zipfile.ZipFile(zip_file, 'r') as zf:
        zf.extractall(destination)

unzip(zip_file='images.zip', destination='./')
```

In [11]:
```
# import matplotlib.pyplot as plt
# from matplotlib.cbook import get_sample_data
#
# def get_image_channels(image_file:str):
#     image = plt.imread(image_file)
#
#     titles = ['Original', 'Red channel', 'Green channel', 'Blue channel']
#     cmaps = [None, plt.cm.Reds_r, plt.cm.Greens_r, plt.cm.Blues_r]
```

```
#
#     fig, axes = plt.subplots(1, 4, figsize=(13,3))
#     objs = zip(axes, (image, *image.transpose(2,0,1)), titles, cmaps)
#
#     for ax, channel, title, cmap in objs:
#         ax.imshow(channel, cmap=cmap)
#         ax.set_title(title)
#         ax.set_xticks(())
#         ax.set_yticks(())
#
#     # plt.savefig('RGB1.png')
#     plt.show()
# image_file='/Users/greysonnewton/Library/CloudStorage/OneDrive-UniversityO
# get_image_channels(image_file)
```

In [12]:

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

def compute_channel_averages(image_dir: str):
    """
    Computes average pixel values for R, G, B channels across all images in

    :param image_dir: Directory path where images are stored.
    :return: Tuple of 3 lists: (red_means, green_means, blue_means)
    """
    red_means, green_means, blue_means = [], [], []

    for file in os.listdir(image_dir):
        if file.lower().endswith(('.jpg', '.jpeg', '.png')):
            path = os.path.join(image_dir, file)
            image = cv2.imread(path)
            if image is None:
                continue
            b, g, r = cv2.split(image)
            red_means.append(np.mean(r))
            green_means.append(np.mean(g))
            blue_means.append(np.mean(b))

    return red_means, green_means, blue_means


def plot_channel_histograms(red_means, green_means, blue_means, bins=30):
    """
    Plots histograms of average pixel values for R, G, B channels.

    :param red_means: List of average red channel values.
    :param green_means: List of average green channel values.
    :param blue_means: List of average blue channel values.
    :param bins: Number of bins in histogram.
    """
    plt.figure(figsize=(16, 4))

    plt.subplot(1, 3, 1)
    plt.hist(red_means, bins=bins, color='red', edgecolor='black')
```

```
    plt.title('Red Channel Averages')
    plt.xlabel('Average Pixel Value')
    plt.ylabel('Frequency')

    plt.subplot(1, 3, 2)
    plt.hist(green_means, bins=bins, color='green', edgecolor='black')
    plt.title('Green Channel Averages')
    plt.xlabel('Average Pixel Value')

    plt.subplot(1, 3, 3)
    plt.hist(blue_means, bins=bins, color='blue', edgecolor='black')
    plt.title('Blue Channel Averages')
    plt.xlabel('Average Pixel Value')

    plt.tight_layout()
    plt.show()


# plot_channel_histograms(*compute_channel_averages(image_dir))
```

In [13]:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

class ImageManipulator:

    color_map = {
        'rgb': cv2.COLOR_BGR2RGB,
        'gray': cv2.COLOR_BGR2GRAY,
    }

    def __init__(self):
        pass

    @classmethod
    def center_crop(cls, image_file: str, crop_factor: float = 0.5):
        if crop_factor > 1 or crop_factor <= 0:
            raise ValueError("Crop factor must be between 0 and 1.")
        image = cv2.imread(image_file)
        if image is None:
            raise FileNotFoundError(f"Unable to read image: {image_file}")
        height, width = image.shape[:2]
        new_width = int(width * crop_factor)
        new_height = int(height * crop_factor)
        left = (width - new_width) // 2
        top = (height - new_height) // 2
        right = left + new_width
        bottom = top + new_height
        return image[top:bottom, left:right]

    @classmethod
    def extract_channels(cls, image_file: str = None, image=None):
        if image is None:
            if image_file is None:
                raise ValueError("No image provided.")
            image = cv2.imread(image_file)
```

```
        if image is None:
            raise FileNotFoundError(f"Unable to read image: {image_file}")
        blue, green, red = cv2.split(image)
        grayscale = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        return {
            'grayscale': grayscale,
            'blue': blue,
            'green': green,
            'red': red
        }

    @classmethod
    def display_image(cls, image: np.ndarray, color_key: str = 'rgb', title:
        if color_key not in (None, 'rgb', 'gray'):
            raise ValueError("color_key must be one of: None, 'rgb', 'gray'"
        if color_key == 'rgb' and len(image.shape) == 3:
            image = cv2.cvtColor(image, cls.color_map['rgb'])
            plt.imshow(image)
        elif color_key == 'gray' or len(image.shape) == 2:
            plt.imshow(image, cmap='gray')
        else:
            plt.imshow(image)
        plt.axis('off')
        plt.title(title)
        plt.show()

    @classmethod
    def display_images(cls, images: dict, ncols: int = 3, figsize: tuple = (
        """
        Displays a stylized subplot of multiple images with an optional mair

        :param images: Dictionary where each key is an ID (not shown), and v
                       {'image': np.ndarray, 'color': 'rgb' or 'gray', 'titl
        :param ncols: Number of columns in the subplot grid.
        :param figsize: Size of the overall figure.
        :param main_title: Optional main title shown above all subplots.
        """
        n_images = len(images)
        nrows = int(np.ceil(n_images / ncols))
        fig, axes = plt.subplots(nrows, ncols, figsize=figsize)

        if nrows == 1:
            axes = np.expand_dims(axes, 0)
        axes = axes.flatten()

        for i, (key, cfg) in enumerate(images.items()):
            img = cfg['image']
            color_key = cfg.get('color', 'rgb')
            title = cfg.get('title', key)

            if color_key == 'rgb' and len(img.shape) == 3:
                img = cv2.cvtColor(img, cls.color_map['rgb'])
                axes[i].imshow(img)
            elif color_key == 'gray' or len(img.shape) == 2:
                axes[i].imshow(img, cmap='gray')
            else:
```

```
                axes[i].imshow(img)

            axes[i].set_title(title, fontsize=12, fontweight='bold')
            axes[i].axis('off')

        # Hide unused subplots
        for j in range(i + 1, len(axes)):
            axes[j].axis('off')

        # Add a big figure title
        fig.suptitle(main_title, fontsize=16, fontweight='bold', y=1.02)
        plt.tight_layout()
        plt.subplots_adjust(top=0.88)  # Adjust spacing under main title
        plt.show()

    @classmethod
    def visualize_color_channel(cls, channel: np.ndarray, color: str):
        blank = np.zeros_like(channel)
        if color == 'red':
            return cv2.merge([blank, blank, channel])
        elif color == 'green':
            return cv2.merge([blank, channel, blank])
        elif color == 'blue':
            return cv2.merge([channel, blank, blank])
        else:
            raise ValueError("Color must be one of 'red', 'green', or 'blue'
```

In [14]:

```
crop_factor = 0.5

def view_image_channels(image_dir:str, n_images:int=3):
    for file in os.listdir(image_dir)[:n_images]:
        if file.lower().endswith(('.jpg', '.jpeg', '.png')):
            image_file = os.path.join(image_dir, file)
            image_filename = image_file.split('/')[-1]
            title = f'Image #{image_filename} Visualization {crop_factor=}'

            original_image = cv2.imread(image_file)
            cropped = ImageManipulator.center_crop(image_file, crop_factor)
            channels = ImageManipulator.extract_channels(image=cropped)

            # Color visualizations
            red_vis = ImageManipulator.visualize_color_channel(channels['red
            green_vis = ImageManipulator.visualize_color_channel(channels['g
            blue_vis = ImageManipulator.visualize_color_channel(channels['bl

            # Prepare images with custom titles
            image_set = {
                'img0': {'image': original_image, 'color': 'rgb', 'title': '
                'img1': {'image': cropped, 'color': 'rgb', 'title': f'Croppe
                'img2': {'image': red_vis, 'color': 'rgb', 'title': f'Red Ch
                'img3': {'image': green_vis, 'color': 'rgb', 'title': f'Gree
                'img4': {'image': blue_vis, 'color': 'rgb', 'title': f'Blue
                'img5': {'image': channels['grayscale'], 'color': 'gray', 't
            }

            ImageManipulator.display_images(image_set, ncols=3, figsize=(15,
```

```
    # plt.show()

# view_image_channels(image_dir)
```

In [15]:

```
import cv2
import numpy as np
import os
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import pandas as pd
class VisionSegmenter:
    def __init__(self, n_clusters=3, crop_factor=0.5):
        """
        :param n_clusters: Number of k-means clusters (default: 3)
        :param crop_factor: Crop factor for center cropping (0 < crop_factor
        """
        self.n_clusters = n_clusters
        self.crop_factor = crop_factor

    def center_crop(self, image):
        """Crop center region of the image based on crop_factor"""
        h, w = image.shape[:2]
        new_w = int(w * self.crop_factor)
        new_h = int(h * self.crop_factor)
        left = (w - new_w) // 2
        top = (h - new_h) // 2
        return image[top:top + new_h, left:left + new_w]

    def load_and_crop(self, image_path):
        """Load an image and apply center crop"""
        image = cv2.imread(image_path)
        if image is None:
            raise FileNotFoundError(f"Unable to read image: {image_path}")
        return self.center_crop(image)

    def segment_image(self, image):
        """Applies k-means to segment the image into clusters"""
        image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        h, w, _ = image_rgb.shape
        pixels = image_rgb.reshape((-1, 3)).astype(np.float32)

        kmeans = KMeans(n_clusters=self.n_clusters, random_state=42, n_init=
        kmeans.fit(pixels)

        clustered = kmeans.cluster_centers_[kmeans.labels_]
        segmented = clustered.reshape((h, w, 3)).astype(np.uint8)
        labels = kmeans.labels_.reshape((h, w))

        return segmented, labels


    def count_inner_pixels(self, labels):
        """
        Counts the total number of pixels for the two smallest clusters
        (assumed to be damaged + impression).
```

```
        :param labels: 2D label map from k-means.
        :return: (inner_pixel_count, cluster_sizes)
        """
        unique, counts = np.unique(labels, return_counts=True)
        cluster_sizes = {int(k): int(v) for k, v in zip(unique, counts)}
        sorted_clusters = sorted(cluster_sizes.items(), key=lambda x: x[1])
        inner_pixel_count = sorted_clusters[0][1] + sorted_clusters[1][1]
        return inner_pixel_count, cluster_sizes

    def estimate_crack_length(self, inner_pixel_count, pixel_size_um=0.5):
        """
        Estimates crack length as the distance from the center to a vertex
        of an equilateral triangle with equivalent area.

        This matches the homework definition:
        crack length = (a * sqrt(3)) / 3, where a is the side length of the
        """
        # Step 1: Estimate side length of equivalent triangle
        side_pixels = np.sqrt((4 * inner_pixel_count) / np.sqrt(3))

        # Step 2: Crack length = center to vertex
        center_to_vertex_pixels = (side_pixels * np.sqrt(3)) / 3

        # Step 3: Convert to microns
        crack_length_um = center_to_vertex_pixels * pixel_size_um
        return crack_length_um

    def calculate_fracture_toughness(self, E, H, P, crack_length_um):
        """
        Calculates fracture toughness Kc.

        :param E: Elastic modulus (GPa)
        :param H: Hardness (GPa)
        :param P: Max load (N)
        :param crack_length_um: Crack length (in microns)
        :return: Fracture toughness (MPa·m^0.5)
        """
        c_m = crack_length_um * 1e-6  # convert microns to meters

        if c_m <= 0:
            raise ValueError("Crack length must be positive.")

        Kc = ((E / H) ** 0.5) * (P / (c_m ** 1.5))
        return Kc

    def visualize(self, original, segmented, labels, title=None):
        """Displays original, segmented image, and label map"""
        plt.figure(figsize=(15, 5))

        plt.subplot(1, 3, 1)
        plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB))
        plt.title("Original")
        plt.axis('off')

        plt.subplot(1, 3, 2)
        plt.imshow(segmented)
```

```
        plt.title("Segmented (K-Means)")
        plt.axis('off')

        plt.subplot(1, 3, 3)
        plt.imshow(labels, cmap='viridis')
        plt.title("Label Mask")
        plt.axis('off')

        if title:
            plt.suptitle(title, fontsize=14, fontweight='bold')

        plt.tight_layout()
        plt.show()

def get_sample_data(df, sample_name:str) -> pd.Series:
    return df[df['image_bank_number'] == int(sample_name)]


def process_image_directory(
        image_dir:str,
        data_file:str,
        max_images=-1,
        n_clusters=3,
        crop_factor=0.5,
        output_dir:str='./output',
        output_name:pd.Timestamp=pd.Timestamp.now().strftime('%Y%m%d%H%M%S')
):
    """
    Processes all images in the directory with k-means segmentation.

    :param image_dir: Path to folder containing images
    :param max_images: Limit for demonstration purposes
    """
    visual_cnt = 3
    dataset = pd.read_excel(data_file)
    # dataset['image_bank_number'] = dataset['image_bank_number'].astype(str
    plot_channel_histograms(*compute_channel_averages(image_dir))
    view_image_channels(image_dir, visual_cnt)

    segment_plt_cnt = 0
    segmenter = VisionSegmenter(n_clusters=n_clusters, crop_factor=crop_fact
    results = []
    for file in os.listdir(image_dir)[:max_images]:
        if not file.lower().endswith(('.jpg', '.jpeg', '.png')):
            continue

        image_path = os.path.join(image_dir, file)
        image_filename = os.path.splitext(file)[0]
        try:
            cropped = segmenter.load_and_crop(image_path)
            segmented, labels = segmenter.segment_image(cropped)

            inner_pixels, cluster_sizes = segmenter.count_inner_pixels(label
            # print(f"{file} - Inner Pixel Count: {inner_pixels} | Cluster S

            crack_length = segmenter.estimate_crack_length(inner_pixels)
```

```
            # print(f"Estimated Crack Length: {crack_length:.2f} µm")

            # print(f'{image_filename=}')
            sample_data = get_sample_data(dataset, image_filename)
            Kc = segmenter.calculate_fracture_toughness(
                sample_data['E(Gpa)'].iloc[0],
                sample_data['H(Gpa)'].iloc[0],
                sample_data['Load(mN)'].iloc[0] / 1000,
                crack_length
            )
            # print(f"Fracture Toughness: {Kc:.4f}")
            # Store result
            results.append({
                'image': file,
                'E(GPa)': sample_data['E(Gpa)'].iloc[0],
                'H(GPa)': sample_data['H(Gpa)'].iloc[0],
                'Load(N)': sample_data['Load(mN)'].iloc[0] / 1000,
                'InnerPixels': inner_pixels,
                'CrackLength(um)': crack_length,
                'FractureToughness(Kc)': Kc
            })

            if segment_plt_cnt < visual_cnt:
                segmenter.visualize(cropped, segmented, labels, title=f"Segm
            segment_plt_cnt += 1

        except Exception as e:
            print(f"Failed on {file}: {e}")
    # Save results to CSV
    results_df = pd.DataFrame(results)
    csv_path = os.path.join(output_dir, f'{output_name}.csv')
    results_df.to_csv(csv_path, index=False)
    print(f"\nResults saved to: {csv_path}")
    return csv_path

image_dir = '/Users/greysonnewton/Library/CloudStorage/OneDrive-UniversityOf
data_file = '/Users/greysonnewton/Library/CloudStorage/OneDrive-UniversityOf
output_dir = '/Users/greysonnewton/Library/CloudStorage/OneDrive-UniversityC
output_file = process_image_directory(image_dir, data_file, output_dir=outpu
pd.read_csv(output_file)
```

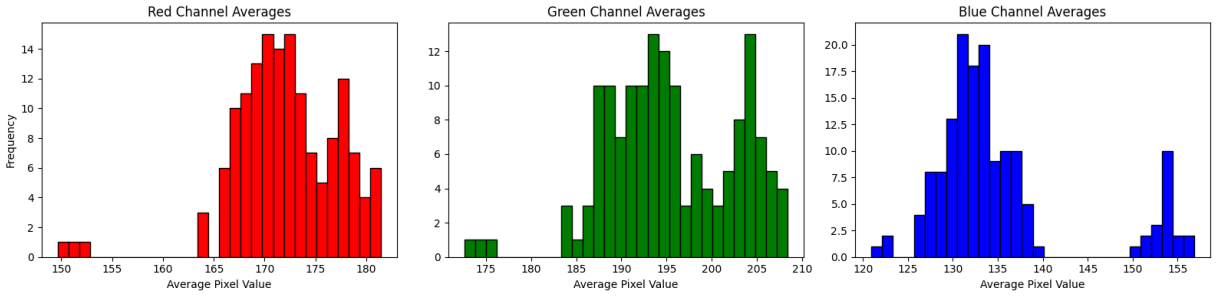


Image #63.JPG Visualization crop_factor=0.5

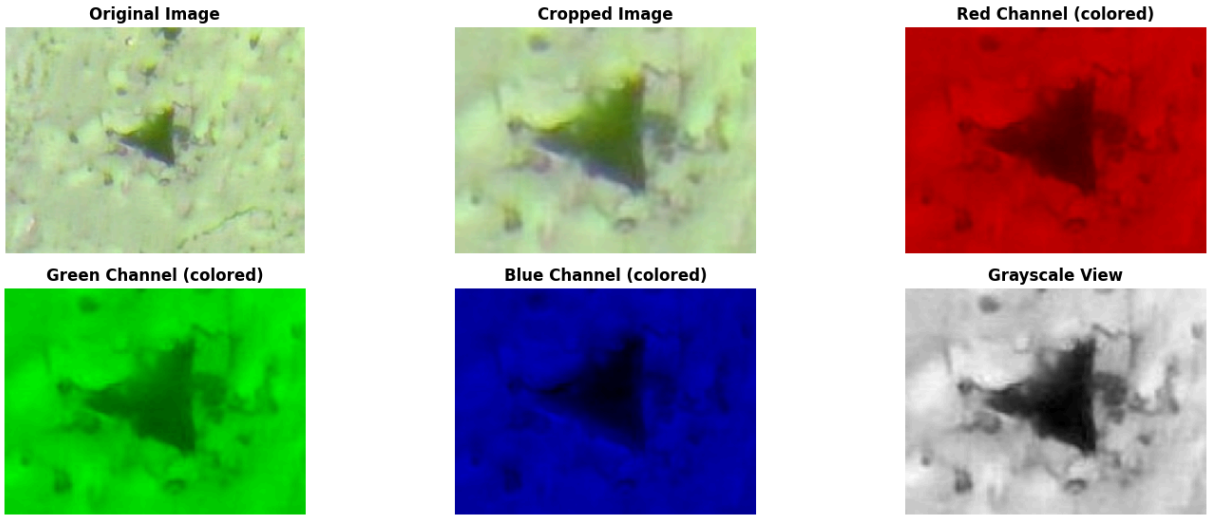


Image #77.JPG Visualization crop_factor=0.5

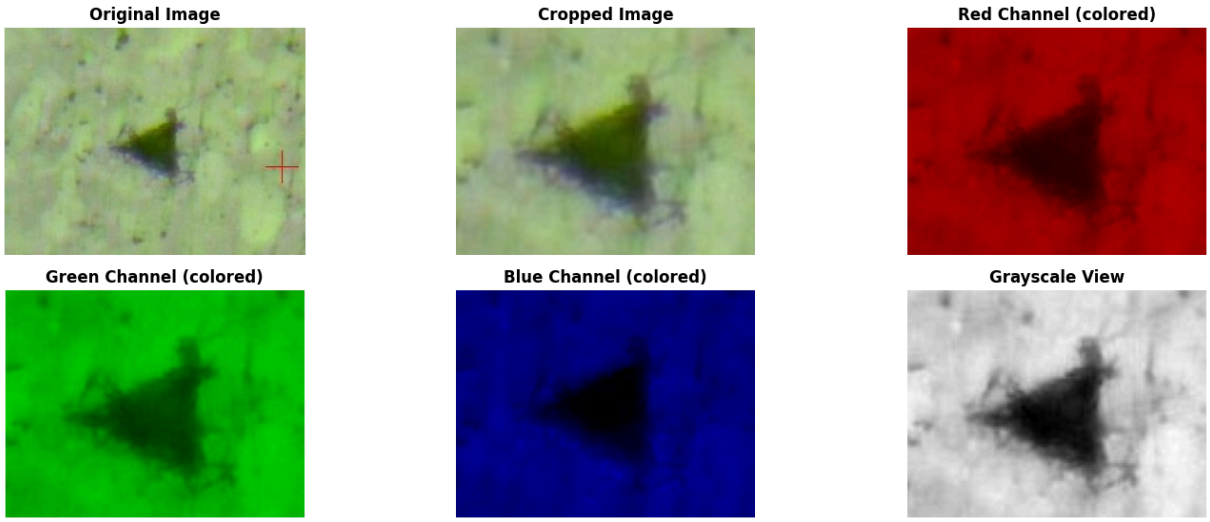
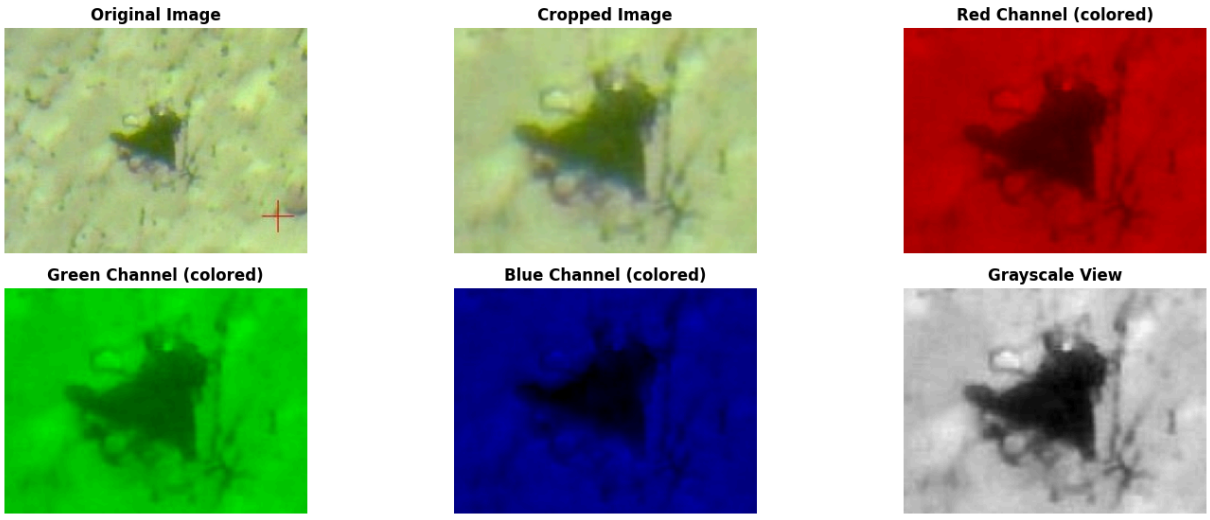
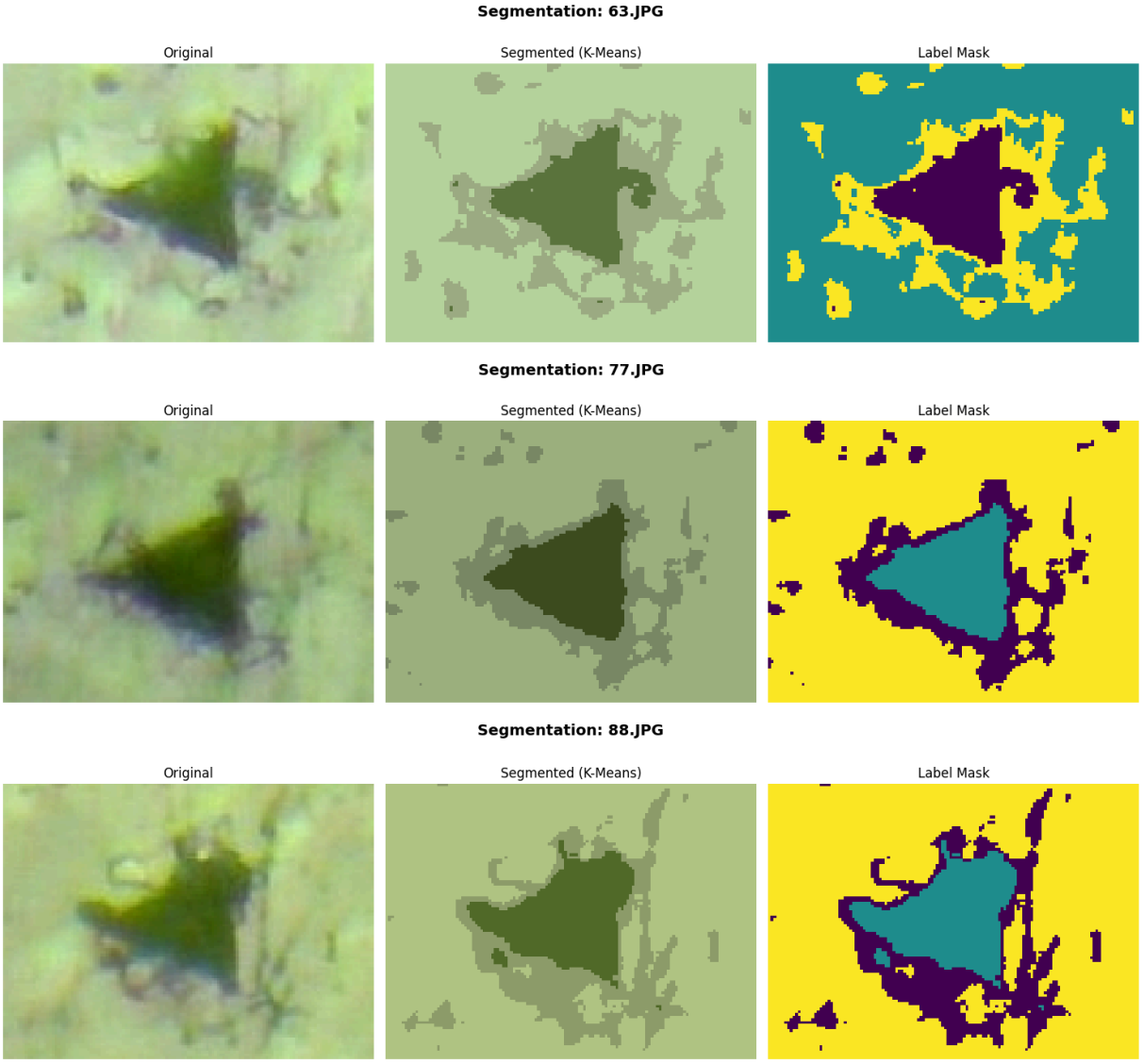


Image #88.JPG Visualization crop_factor=0.5






```
Results saved to: /Users/greysonnewton/Library/CloudStorage/OneDrive-Univers
ityOfHouston/FALL_2025/Big Data Analytics (PETR)/big-data-analytics/big-data
-analytics/homeworks/hw5/output/20250410215903.csv
```

Out[15]:

	image	E(GPa)	H(GPa)	Load(N)	InnerPixels	CrackLength(um)	FractureTough
0	63.JPG	79.3	2.13	0.399132	3976	27.661915	1.67
1	77.JPG	72.1	2.44	0.449432	3207	24.843257	1.97
2	88.JPG	50.1	1.12	0.449906	3891	27.364636	2.10
3	89.JPG	68.1	2.33	0.449860	4065	27.969798	1.64
4	76.JPG	69.2	2.03	0.449484	3331	25.318990	2.05
...	...	...	...	...	...	...	
144	90.JPG	63.5	2.22	0.449090	3719	26.752979	1.73
145	91.JPG	70.2	2.39	0.449474	3189	24.773440	1.97
146	85.JPG	69.2	2.69	0.449902	4163	28.304942	1.51
147	147.JPG	62.3	2.34	0.598946	4378	29.026651	1.97
148	52.JPG	59.7	2.22	0.349305	4201	28.433832	1.19

149 rows × 7 columns

In [ ]:

In [ ]: